

QUICK-REFERENCE CHEAT SHEET

12-Week Technology Career Discovery Programme

All resource links, plus the Linux, Git, Python, networking and Markdown commands you'll actually reach for.
Keep this open in a tab or print it and keep it next to you.

All Resource Links

One primary link per week — check it's still current before the week starts (free platforms change their offerings over time).

Wk	Resource	Link
1–2	Linux Journey	linuxjourney.com
1–2	MIT Missing Semester	missing.csail.mit.edu
1–2	freeCodeCamp Linux (backup)	freecodecamp.org
3–4	Python Essentials 1 & 2 (Cisco/OpenEDG)	netacad.com
3–4	Official Python tutorial (backup)	docs.python.org/3/tutorial
3–4	Automate the Boring Stuff	automatetheboringstuff.com
5	GitHub Skills	skills.github.com
5	Atlassian Git tutorials (backup)	atlassian.com/git
6	Cisco Networking Academy — Intro to Networks	netacad.com
6	Professor Messer Network+ (backup)	professormesser.com
7	TryHackMe — Pre Security path	tryhackme.com
7	Cisco Skills for All — Intro to Cybersecurity (backup)	netacad.com
8	Microsoft Learn — Azure Fundamentals	learn.microsoft.com
8	AWS Skill Builder (backup)	skillbuilder.aws
9	The Odin Project	theodinproject.com
9	freeCodeCamp backend (backup)	freecodecamp.org
10	Kaggle Learn	kaggle.com/learn
10	freeCodeCamp Data Analysis (backup)	freecodecamp.org
11	Random Nerd Tutorials	randomnerdtutorials.com
11	Arduino / Espressif docs (backup)	docs.arduino.cc · docs.espressif.com
—	OverTheWire Bandit (stretch, Wks 1–2)	overthewire.org/wargames/bandit
—	GitHub	github.com
—	Wireshark	wireshark.org
—	Cisco Packet Tracer download	netacad.com (via course enrolment)
—	draw.io (free diagrams)	app.diagrams.net

Linux / Terminal Cheat Sheet

Navigation & files

Command / syntax	What it does
<code>pwd</code>	Print the current directory
<code>ls -la</code>	List all files, including hidden ones, with details
<code>cd folder/</code>	Change directory
<code>cd ..</code>	Move up one directory
<code>mkdir name</code>	Create a new folder
<code>touch file.txt</code>	Create a new empty file
<code>cp source dest</code>	Copy a file
<code>mv source dest</code>	Move or rename a file
<code>rm file</code>	Delete a file (careful — no recycle bin)
<code>rm -r folder/</code>	Delete a folder and everything in it
<code>cat file</code>	Print a file's contents
<code>less file</code>	View a file page by page (q to quit)
<code>find . -name "*.py"</code>	Find files matching a pattern
<code>grep "text" file</code>	Search for text inside a file

Permissions & ownership

Command / syntax	What it does
<code>chmod 755 file</code>	Set read/write/execute permissions (owner/group/others)
<code>chmod +x script.sh</code>	Make a file executable
<code>chown user:group file</code>	Change who owns a file
<code>ls -l</code>	Show permissions alongside file listing

Processes & system

Command / syntax	What it does
<code>ps aux</code>	List running processes
<code>top</code>	Live view of running processes (q to quit)
<code>kill PID</code>	Stop a process by its ID
<code>df -h</code>	Show disk usage, human-readable
<code>du -sh folder/</code>	Show a folder's total size
<code>echo \$PATH</code>	Show the PATH environment variable
<code>env</code>	List all environment variables
<code>history</code>	Show your recent command history

Package management (Ubuntu/Debian)

Command / syntax	What it does
<code>sudo apt update</code>	Refresh the list of available packages
<code>sudo apt install name</code>	Install a package
<code>sudo apt remove name</code>	Remove a package
<code>apt search name</code>	Search for a package

SSH

Command / syntax	What it does
<code>ssh-keygen -t ed25519</code>	Generate a new SSH key pair
<code>ssh user@host</code>	Connect to a remote machine
<code>ssh-copy-id user@host</code>	Copy your public key to a remote machine for key-based login
<code>scp file user@host:path</code>	Copy a file to a remote machine

Git & GitHub Cheat Sheet

Command / syntax	What it does
<code>git init</code>	Turn the current folder into a Git repository
<code>git status</code>	See what's changed since the last commit
<code>git add file</code>	Stage a file for the next commit
<code>git add .</code>	Stage everything that's changed
<code>git commit -m "message"</code>	Save a snapshot with a message
<code>git log --oneline</code>	See commit history, one line per commit
<code>git branch name</code>	Create a new branch
<code>git checkout -b name</code>	Create and switch to a new branch
<code>git switch name</code>	Switch to an existing branch
<code>git merge name</code>	Merge a branch into the current one
<code>git diff</code>	See exactly what's changed, line by line
<code>git remote add origin URL</code>	Connect a local repo to a GitHub repo
<code>git push -u origin main</code>	Push your commits to GitHub for the first time
<code>git push</code>	Push new commits after that
<code>git pull</code>	Pull down the latest changes
<code>git clone URL</code>	Copy a remote repo to your machine

When you get a merge conflict

Git will mark the conflicting section in the file with <<<<<<, =====, and >>>>>>

Edit the file by hand to keep what you want and delete the markers

git add the fixed file, then git commit to finish the merge

Good .gitignore starters (Python projects)

Command / syntax	What it does
<code>__pycache__/</code>	Skip Python's compiled bytecode cache
<code>*.pyc</code>	Skip individual compiled files
<code>.env</code>	Never commit secrets/API keys
<code>venv/</code>	Skip your local virtual environment folder

Python Quick Syntax

Basics

Command / syntax	What it does
<code>x = 5</code>	Variable assignment
<code>type(x)</code>	Check a variable's type
<code>print(f"{x}")</code>	f-string formatting
<code>input("prompt: ")</code>	Read user input (always returns a string)

Control flow

Command / syntax	What it does
<pre>if x > 5: ... elif x == 5: ... else: ...</pre>	Conditional branches
<pre>for item in items: ...</pre>	Loop over a list/collection
<pre>for i in range(10): ...</pre>	Loop a fixed number of times
<pre>while condition: ...</pre>	Loop while something is true
<code>break / continue</code>	Exit a loop early / skip to the next iteration

Functions & collections

Command / syntax	What it does
<pre>def name(a, b): return a + b</pre>	Define a function
<code>[1, 2, 3]</code>	List — ordered, changeable
<code>(1, 2, 3)</code>	Tuple — ordered, unchangeable
<code>{"key": "value"}</code>	Dictionary — key/value pairs
<code>{1, 2, 3}</code>	Set — unordered, unique values
<code>items.append(x)</code>	Add to a list
<code>d.get("key", default)</code>	Safely read a dict value
<code>[x*2 for x in items]</code>	List comprehension

Files & exceptions

Command / syntax	What it does
<pre>with open("f.txt") as f: data = f.read()</pre>	Read a file safely (auto-closes)

Command / syntax	What it does
<code>with open("f.txt", "w") as f: f.write(text)</code>	Write to a file
<code>import csv / import json</code>	Work with CSV or JSON files
<code>try: ... except Exception as e: ...</code>	Catch and handle errors

Classes & packages

Command / syntax	What it does
<code>class Item: def __init__(self, name): self.name = name</code>	Define a class
<code>item = Item("apple")</code>	Create an object
<code>import module_name</code>	Import a module
<code>pip install package</code>	Install a third-party package
<code>pip freeze > requirements.txt</code>	Save your installed packages list

Networking Quick Reference

OSI model (top to bottom)

Command / syntax	What it does
7 — Application	HTTP, DNS, FTP, SSH — what the user-facing software speaks
6 — Presentation	Encryption, formatting (TLS/SSL lives roughly here)
5 — Session	Managing connections/sessions between apps
4 — Transport	TCP (reliable) and UDP (fast, no guarantee)
3 — Network	IP addressing and routing
2 — Data Link	MAC addresses, switches
1 — Physical	Cables, radio waves, the actual signal

Common ports

Command / syntax	What it does
20 / 21	FTP (data / control)
22	SSH
23	Telnet (insecure — avoid)
25	SMTP (email sending)
53	DNS
67 / 68	DHCP
80	HTTP
443	HTTPS
3389	RDP

Private IPv4 ranges (won't route on the public internet)

Command / syntax	What it does
10.0.0.0 – 10.255.255.255	10.0.0.0/8
172.16.0.0 – 172.31.255.255	172.16.0.0/12
192.168.0.0 – 192.168.255.255	192.168.0.0/16

Subnetting cheat: /prefix → hosts

Command / syntax	What it does
/24	256 addresses (254 usable) — 255.255.255.0
/25	128 addresses (126 usable) — 255.255.255.128
/26	64 addresses (62 usable) — 255.255.255.192
/27	32 addresses (30 usable) — 255.255.255.224
/28	16 addresses (14 usable) — 255.255.255.240

Command / syntax	What it does
/29	8 addresses (6 usable) — 255.255.255.248
/30	4 addresses (2 usable) — 255.255.255.252

Markdown (for READMEs) & Nmap Basics

Markdown syntax

Command / syntax	What it does
# Heading	Level-1 heading (## for level 2, etc.)
bold / <i>*italic*</i>	Bold / italic text
- item	Bullet list
1. item	Numbered list
`code`	Inline code
``` code block ```	Multi-line code block
[text](https://url)	A link
![alt text](image.png)	An image
> quote	Blockquote

## Nmap — local VM only, never against a system you don't own

Command / syntax	What it does
nmap 192.168.1.1	Basic scan of a single host
nmap -sV 192.168.1.1	Detect service versions
nmap -p 1-1000 192.168.1.1	Scan a specific port range
nmap -A 192.168.1.1	Aggressive scan (OS detection, versions, scripts)

A reminder from the AI & integrity policy: running these against anything other than your own VM, or a machine you have explicit written permission to test, is both illegal and against the programme's ethics section. Keep every scan local.

*Keep this next to you. You'll use the Linux and Git pages every week of the programme.*